NORDIC MACHINE INTELLIGENCE

Nordic Machine Intelligence, vol. x, pp. 1–17, 20xx
Received xx Jan 2026 / published xx Jan 20xx
https://doi.org/10.5617/nmi.some-doi

Graph Embeddings for Software Architecture Recovery

Rakhshanda Jabeen[12] Morgan Ericsson[1], Jonas Nordqvist[3], and Anna Wingkvist[1]

1. *Department of Computer Science, Linnaeus University, Växjö, Sweden*
2. *Electrolux Professional AB, Ljungby, Sweden*
3. *Department of Mathematics, Linnaeus University, Växjö, Sweden*

Abstract

A software system's source code is organized into files, but the way those files group into architectural modules is often undocumented or has drifted from the original design. Software architecture recovery (SAR) reconstructs this module view to help developers understand and maintain large systems.

We study SAR from file-level static dependency graphs by learning file embeddings in a self-supervised manner and clustering them into modules. Using node2vec as a baseline, we train graph autoencoders via link reconstruction and compare dependency-based representations (a graph attention encoder, with and without personalized PageRank diffusion) against heterogeneous ones that also incorporate the project's folder structure. We evaluate eleven open-source Java, C, and C++ systems against ground-truth architectures using complementary alignment and modularity metrics. Recovery from a dependency structure alone is useful but limited, bounded more by how much structural context is available to the model than by the choice of encoder. Folder organization helps where dependency cohesion is weak, but heterogeneous methods often gain alignment by drifting from pure dependency modularity, a trade-off between structural cohesion and the developer-intended decomposition. No single method wins on every system and metric — node2vec and the diffusion model are comparable on dependencies alone, while folder-aware methods are worth their added complexity mainly when cohesion is weak. All representations scale well and are practical to run and tune, so representation learning is not the bottleneck for adoption.

Keywords: Variational graph autoencoders, Graph attention networks, Software architectures

1 Introduction

Software architecture describes the high-level structure of a system in terms of its principal elements, their responsibilities, and the relations among them [1]. In implementation, this structure is often reflected as a modular decomposition: a partition of files or classes into modules that group related functionality [2]. Such decompositions are important for program understanding, maintenance, onboarding, and refactoring, but documented architecture is often missing, incomplete, or quickly outdated as systems evolve [1]. The problem is becoming more pressing as AI coding assistants increase the volume of code they generate, with the intended module structure seldom recorded.

Software Architecture Recovery (SAR) aims to infer this modular decomposition directly from implementation artifacts [3, 4]. Traditional SAR methods often rely on the software dependency graph (SDG), where files or classes are connected by static relations such as calls, imports, and inheritance. The underlying assumption is that files implementing related functionality tend to be more densely connected within modules than across module boundaries [5]. Existing approaches exploit this signal through hierarchical clustering [6, 7], search-based optimization guided by cohesion and coupling objectives [8, 9, 10], or complementary sources such as lexical semantics [11, 12], multi-source fusion [13, 14], and large language models [15].

Graph representation learning provides another way to use these signals. Instead of directly clustering the SDG, it learns dense embeddings of source files and clusters the resulting representations. NEGAR [5] showed that node2vec-style random walks can recover useful architectural structure from dependency context. GAER [4] extended this direction by using Graph Neural Network (GNN) encoders over heterogeneous dependency graphs, typed edges, and semantic node initialization, demonstrating that graph neural models with semantic features can improve recovery. These results leave open an important question: how far can graph representation learning go from dependencies alone, and when are additional signals needed?

We study graph representation learning for SAR by separating the contributions of dependency structure and

project organization. First, we evaluate dependency-only representations on the homogeneous file-level SDG, including random-walk embeddings and Graph Attention Network (GAT) [16] encoders trained within a Variational Graph Autoencoder (VGAE) [17] objective. In this setting, the GAT learns file embeddings from dependency neighborhoods, and the VGAE objective trains the embeddings to reconstruct dependency edges. Second, we augment the SDG with folder nodes and folder–file containment edges to test whether project layout provides an architectural signal beyond dependencies. We compare meta-path-guided random walks with heterogeneous GAT encoders, and include a typed-dependency variant to test whether preserving dependency types adds value within the heterogeneous setting. Finally, we evaluate the practical cost of these methods in terms of runtime and hyperparameter sensitivity[1].

Our empirical study is guided by the following research questions:

RQ1: How effectively can software architecture be recovered from dependency structure alone?

RQ2: Does augmenting the dependency graph with project organizational structure improve recovery?

RQ3: What are the practical costs and tuning sensitivities of these methods as system size grows?

The paper is organized as follows: Section 2 reviews related work, and Section 3 formulates the learning objective and describes our methodology. Section 4 details the experimental configurations and evaluation metrics, and Section 5 presents and analyzes the results. Section 6 discusses threats to validity, and Section 7 concludes the paper and outlines future directions.

2 Related Work

Prior SAR work leverages different signals, including static dependencies [18, 19, 8], project organization, such as directory structure and file paths [20], textual information, such as identifiers and comments [21, 22], and hybrid approaches that integrate multiple signals [23, 24, 25].

SDG-based approaches use structural information for modularization and fall into two groups. Search-based methods cast recovery as the optimization of a modularization objective, such as Modularization Quality (MQ), which rewards cohesion and penalizes coupling. Bunch and its hill-climbing variants (NAHC/SAHC) are representative [9], and MCA and ECA add multi-objective genetic optimization [26]. Hierarchical methods instead merge clusters greedily by dependency-derived similarity: WCA uses dependency-based feature vectors [10], LIMBO summarizes the SDG using information theory before merging [7], and CCT [27] and HCC [6] combine multiple similarity views or dendrograms into a consensus decomposition. Altınışık et al. [28] proposed an approach that treats each dependency model separately during clustering and then aggregates the resulting decompositions. A further line targets scalability, including FCA's matrix-based clustering for large systems [29], the deterministic neighborhood-tree method of Mohammadi and Izadkhah [30], and GMA, which derives similarity from the depth of dependency relations [31].

Beyond dependencies, many approaches used directory structure and semantic signals from code and documentation for SAR [20]. ARC [11] uses topic modeling, while ZBR [12] constructs weighted lexical zones via expectation maximization on textual features to support clustering. Hybrid methods combine structural and non-structural sources using automated feature weighting [32]; SADE and SARIF fuse multiple signals by constructing weighted dependency graphs and applying community detection [14, 13]. More recently, LLM-assisted approaches have explored mapping implementation units to a reference architecture using chain-of-thought prompting and deductive reasoning [15]. SemArc [33] uses an LLM to interpret code and architectural semantics based on canonical patterns, then clusters components using an anchor-based strategy.

Representation learning offers an alternative: it learns dense embeddings of source-code files and clusters them to recover modules. NEGAR [5] uses node2vec-style random walks with a Skip-gram objective to place files with similar structural contexts close together, then clusters the embeddings. GAER [4] extended this with GNN encoders, comparing GCN and GAT encoders within a graph autoencoder. It models each dependency type as a separate relation and combines structural dependencies with semantic signals from source code, such as file paths and code identifiers, and reports improved recovery over node2vec and state-of-the-art SAR methods. Jabeen et al. [34] used heterogeneous GCNs for a closely related task, mapping source-code files to architectural modules.

Learning embeddings that reconstruct graph structure and then partitioning them is a common clustering approach well beyond SAR, used in community detection [35], financial risk analysis [36], and time-series clustering [37]. GAER applies this approach to SAR, combining dependency, organizational, and semantic signals without separating them. Building on this line of work, we fix the model, a GAT encoder trained within a VGAE, and vary only the input graph: the file-level SDG alone, or with folder nodes and containment edges added. This isolates what dependencies recover on their own from what organizational structure contributes.

3 Methodology

This section describes the architecture-recovery pipeline used in the study. The pipeline first constructs a graph from source-code dependencies and, where applicable,

[1]Source code and data are available at github.com/sse-lnu/StructSAR.

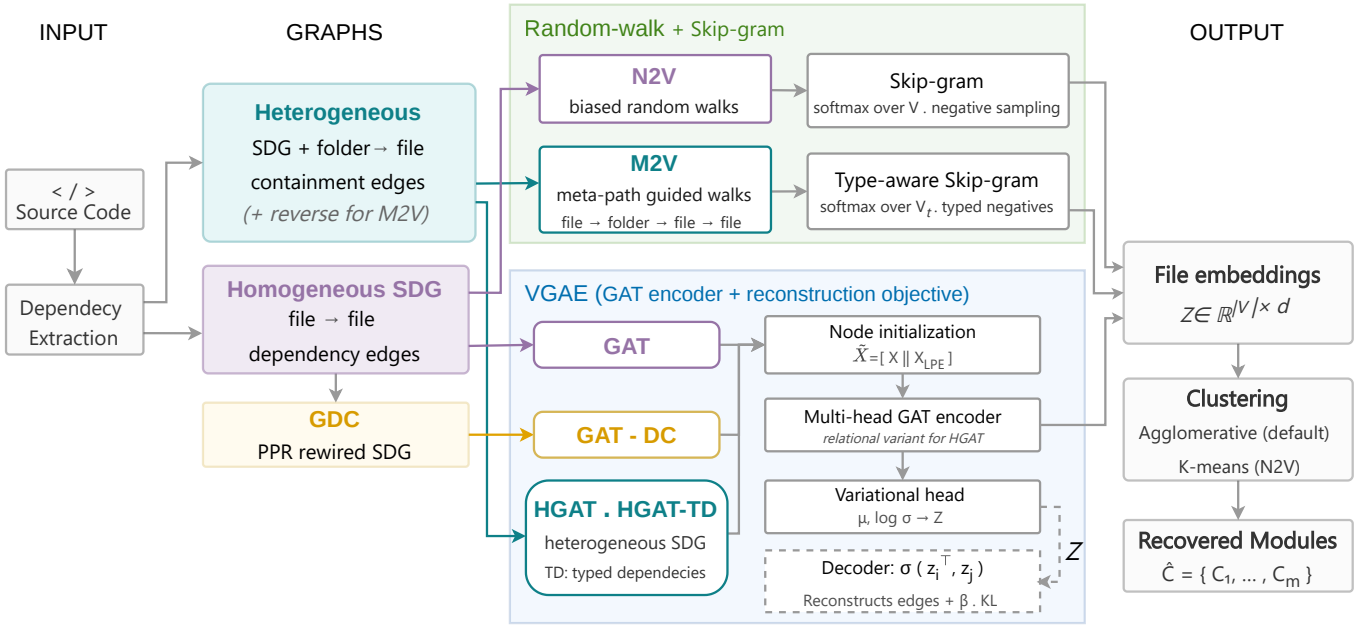


Figure 1: Schematic workflow of studied methods. Static file-level dependencies are extracted from source code and, for the heterogeneous methods, augmented with folder nodes and containment edges. The resulting graph feeds one of the representation learners: biased random walks with Skip-gram (N2V), meta-path-guided walks with type-aware Skip-gram (M2V), or a GAT encoder trained within a VGAE objective on the homogeneous graph (GAT), on a PPR-diffused homogeneous graph (GAT-DC), or on the heterogeneous graph with relation-specific message passing (HGAT), with a variant (HGAT-TD) that keeps typed dependency edges. The resulting file embeddings Z are then clustered to recover the architectural decomposition $\hat{\mathcal{C}}$.

folder containment relations. It then learns an embedding for each source-code file and clusters the file embeddings to recover a modular decomposition.

The graph construction follows the first two research questions. For RQ1, we use a homogeneous file-level SDG, in which nodes are files and edges represent static dependencies. For RQ2, we augment this graph with folder nodes and folder–file containment edges, yielding a heterogeneous graph that represents both dependency structure and project organization. We also include a typed-dependency variant of the heterogeneous model, HGAT-TD, which keeps dependency types separate and tests whether typed dependency relations add value within the heterogeneous setting.

The learning methods vary along two axes: the input graph and the representation-learning strategy. The random-walk variants use a Skip-gram objective, while the GAT-based variants use a graph attention encoder trained with a VGAE reconstruction objective. The learned file embeddings are then clustered to recover a modular decomposition. RQ3 evaluates the runtime and tuning sensitivity of the same representations. Figure 1 summarizes the pipeline.

3.1 Graph Construction

Static dependencies capture structural relationships between source-code files. Files that implement related functionality often depend on one another directly, or are connected through short dependency paths, making the dependency graph a useful signal for architecture recovery [5]. Dependency types may also carry different architectural meanings; for example, call edges can indicate tighter coupling than type-usage edges [38]. Project organization provides a complementary signal: folders often reflect how developers decompose a system into responsibilities and group related functionality [39, 13].

We extract static dependencies using Depends [40]. Although Depends can report finer-grained method- or variable-level information, we aggregate dependencies to the file level because the ground-truth architectures used in this study are labeled per file. We construct two graph forms from the same extracted dependencies: a homogeneous dependency graph used for RQ1, and a heterogeneous graph that augments dependencies with folder containment relations for RQ2.

Homogeneous software graph. The homogeneous graph $G = (V, A)$ contains one node per source-code file, so $V = V_{\texttt{file}}$. A single undirected edge connects two files if at least one static dependency exists between them, regardless of dependency type. Formally, $A_{ij} = 1$ if files i and j are connected by one or more dependencies, and $A_{ij} = 0$ otherwise. All dependency types are therefore merged into one file–file relation.

Heterogeneous software graph. The heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}, \mathcal{R})$ augments the file-level dependency graph with project layout information. The node set $\mathcal{V} = V_{\texttt{file}} \cup V_{\texttt{folder}}$ contains both file nodes and folder nodes, where each directory in the source tree is represented as

one folder node. The node-type function $\phi : \mathcal{V} \to \mathcal{T}$ maps each node to its type, with $\mathcal{T} = \{\texttt{file}, \texttt{folder}\}$.

The edge set $\mathcal{E}$ is typed by a relation function $\psi : \mathcal{E} \to \mathcal{R}$. The relation set $\mathcal{R}$ contains:

1. $\texttt{file} \xrightarrow{r} \texttt{file}$: a static dependency of type r;
2. $\texttt{folder} \xrightarrow{contains} \texttt{file}$: a containment relation from a folder to each file it directly contains;
3. $\texttt{file} \xrightarrow{in} \texttt{folder}$: the reverse containment relation, used by M2V walks.

For M2V and HGAT, all file–file dependency types are merged into a single `depends_on` relation matching the homogeneous graph. HGAT-TD is a typed-dependency variant: it keeps the file–file dependency types as separate relations while using the same folder containment structure. Thus, all methods use the same extracted dependencies; they differ in whether they also use folder organization and whether dependency types are merged or preserved.

3.2 Learning Objective

Given the constructed graph $\mathcal{G}$, our goal is to recover an architectural decomposition from its source-code files using graph representation learning. Here, the input graph is G for homogeneous methods and $\mathcal{G}$ for heterogeneous methods. Concretely, we learn an encoder:

$$f_\theta : \mathcal{G} \longrightarrow Z \in \mathbb{R}^{|V_{\texttt{file}}| \times d}, \tag{1}$$

that maps each source-code file $i \in V_{\texttt{file}}$ to a d-dimensional embedding $z_i \in \mathbb{R}^d$, where θ denotes the learnable parameters of the representation-learning method. The resulting file embeddings Z are then clustered to recover the architectural decomposition:

$$\hat{\mathcal{C}} = \{C_1, \ldots, C_m\},$$

a partition of the source-code files into m disjoint modules. Folder nodes, when present, are used only to support representation learning and are not included in the recovered decomposition.

3.3 Random-Walk Based Methods

Random-walk based methods learn file embeddings from node sequences sampled over the graph. Nodes that appear in similar walk contexts are encouraged to have similar embeddings through a Skip-gram objective. We study two variants: Node2Vec (N2V), which samples biased walks on the homogeneous dependency graph, and MetaPath2Vec (M2V), which guides walks over the heterogeneous graph using a meta-path schema. The following subsections describe the two methods.

3.3.1 Node2Vec (N2V) on Homogeneous Graph

We apply node2vec [41] on the homogeneous dependency graph, following the architecture-recovery setup of NEGAR [5]. Node2vec samples biased random walks on G and feeds the resulting node sequences to a Word2Vec-style Skip-gram model [42]. Walk bias is controlled by parameters p and q, which adjust transition probabilities to favor either staying near the starting node or exploring farther regions, so the model can capture both local connectivity and longer-range structural patterns.

Given the generated sequences, Skip-gram forms training pairs by treating nodes within a fixed context window centered on a target node i as observed context nodes j. Let $z_i \in \mathbb{R}^d$ denote the embedding of node i. The model assigns a probability to observing context node j given i using a softmax over dot-product similarities:

$$p(j \mid i) = \frac{\exp(z_i^\top z_j)}{\sum_{k \in V} \exp(z_i^\top z_k)}$$

Full softmax over all nodes is expensive, so training uses negative sampling. For each observed pair (i, j), we sample η negative nodes $\{n_m\}_{m=1}^{\eta}$ (nodes not observed in the context of i) and update embeddings to increase similarity for (i, j) while decreasing similarity for (i, n_m), by maximizing the following objective:

$$\log \frac{1}{1 + e^{-z_i^\top z_j}} + \sum_{m=1}^{\eta} \log \frac{1}{1 + e^{z_i^\top z_{n_m}}}. \tag{2}$$

3.3.2 MetaPath2Vec (M2V) on Heterogeneous Graph

MetaPath2Vec [43] extends random-walk-based embedding to heterogeneous graphs by guiding the walk with a meta-path schema $\mathcal{P}$ – a fixed sequence of allowed node and edge types. We use the file-folder containment links to guide the walker, so the sampled sequences reflect both folder co-membership and file dependency relations. The meta-path alternates between file and folder nodes as:

$$\texttt{file} \xrightarrow{in} \texttt{folder} \xrightarrow{contains} \texttt{file} \xrightarrow{\texttt{depends_on}} \texttt{file}, \ldots$$

At each step, the walker selects the next node uniformly from neighbors that match the required next type in $\mathcal{P}$, and the schema repeats cyclically. Two files therefore appear in the same walk either because they depend on each other, because they share a folder, or through a longer chain combining both relations.

Given the sampled sequences, nodes c_t of type t appearing within a fixed window around target node i form observed context pairs. MetaPath2Vec models their conditional probability using a type-restricted softmax:

$$p(c_t \mid i) = \frac{\exp(z_i^\top z_{c_t})}{\sum_{k \in V_t} \exp(z_i^\top z_k)}, \tag{3}$$

where V_t contains nodes of the same type as c_t. We optimize the negative-sampling objective in Eq. 2, drawing negative nodes from V_t so that each positive pair is contrasted against same-type candidates.

3.4 Graph Attention Network-Based Methods

Random-walk methods learn from sampled sequences and co-occurrence statistics. GNNs take a different route: they compute node representations through message passing, iteratively updating each node from its neighbors [44]. We use the GAT as the encoder and train it within the VGAE framework [17], which reconstructs the observed dependency edges from the learned embeddings, thereby providing a self-supervised training objective.

However, a shallow L-layer GNN can only aggregate information from nodes within L hops. Increasing the number of layers may capture longer-range dependencies, but it also risks over-smoothing, where node representations become less discriminative [45]. We therefore study structural inputs that are prepared before message passing: Laplacian positional encoding (LPE) [46], which augments node features with graph-position information, and Graph Diffusion Convolution (GDC) [47], which rewires the input graph so that diffusion-close nodes become direct neighbors.

3.4.1 Node Initialization

Each variant starts from a trainable embedding table learned with the rest of the model. For the homogeneous variants, a single table $X \in \mathbb{R}^{|V_{\texttt{file}}| \times d_0}$ covers all file nodes, with row x_i giving the initial representation of file i; the heterogeneous variants maintain one table per node type, $X^{(t)} \in \mathbb{R}^{|V_t| \times d_0}$ for $t \in \{\texttt{file}, \texttt{folder}\}$.

A randomly initialized embedding carries no information about where a file sits in the dependency graph, so two files in opposite regions of the system look identical to the encoder before message passing begins. Augmenting GAT inputs with positional information is known to help on graphs where neighbors are not necessarily similar [48]. We therefore inject position information into the initial features via Laplacian positional encoding (LPE) [46], taking the first ζ non-trivial eigenvectors of the symmetric normalized graph Laplacian of the file–file graph as a feature matrix $X_{\mathrm{LPE}} \in \mathbb{R}^{|V_{\texttt{file}}| \times \zeta}$. The encoder input is the row-wise concatenation of the trainable embedding and the LPE features:

$$\tilde{X} = [\, X \parallel X_{\mathrm{LPE}} \,] \in \mathbb{R}^{|V_{\texttt{file}}| \times (d_0 + \zeta)}, \tag{4}$$

where $\parallel$ denotes concatenation. Hence, each file enters the encoder with a position-aware feature $\tilde{x}_i$. For HGAT and HGAT-TD, LPE is computed on the file–file subgraph and applied only to file nodes; folder nodes use their trainable embedding alone.

3.4.2 Encoder

We use GAT [16] to refine the initial representations through message passing on the input graph, assigning a learned importance weight to each neighbor before aggregation. Let $\mathcal{N}(i)$ denote the one-hop neighborhood of node i including a self-loop, and let $h_i^{(\ell)}$ be its representation after ℓ GAT layers, with $h_i^{(0)} = \tilde{x}_i$.

For each neighbor $j \in \mathcal{N}(i)$, GAT computes an unnormalized attention score:

$$e_{ij}^{(\ell)} = \mathrm{LeakyReLU}\Big(a^{(\ell)\top}[W^{(\ell)} h_i^{(\ell)} \parallel W^{(\ell)} h_j^{(\ell)}]\Big), \tag{5}$$

where $\mathrm{LeakyReLU}(\cdot)$ is the leaky rectified linear unit [49], $W^{(\ell)}$ is a learnable linear transformation, and $a^{(\ell)}$ is a learnable attention vector. The final attention coefficient $\alpha_{ij}^{(\ell)}$ is obtained by softmax normalization of $e_{ij}^{(\ell)}$ over all neighbors in $\mathcal{N}(i)$.

For the homogeneous variants (GAT, GAT-DC), each layer updates a node's representation as a weighted aggregation of transformed neighbor features:

$$h_i^{(\ell+1)} = \mathrm{ELU}\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(\ell)} W^{(\ell)} h_j^{(\ell)}\right), \tag{6}$$

where $\mathrm{ELU}(\cdot)$ is the exponential linear unit [50]. We use multi-head attention and average across heads; stacking L layers yields the embedding matrix $H^{(L)} \in \mathbb{R}^{|V| \times d}$.

The heterogeneous variants (HGAT, HGAT-TD) instead perform relation-specific message passing, aggregating over the relations in $\mathcal{R}$:

$$h_i^{(\ell+1)} = \mathrm{ELU}\left(\sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_r(i)} \alpha_{ij}^{(r,\ell)} W_r^{(\ell)} h_j^{(\ell)}\right), \tag{7}$$

where $\mathcal{N}_r(i)$ is the neighborhood of i under relation r, $W_r^{(\ell)}$ is a relation-specific weight matrix, and $\alpha_{ij}^{(r,\ell)}$ are attention weights normalized over $\mathcal{N}_r(i)$.

3.4.3 Structural Augmentation with Graph Diffusion Convolution

Files in the same architectural module are often close in the dependency graph but not directly connected [5], so with only one or two GAT layers, they never exchange messages. GDC [47] addresses this by replacing the raw adjacency A with a diffused adjacency $\tilde{A}$ that links diffusion-close nodes directly and assigns each new edge an importance weight.

We compute $\tilde{A}$ using Personalized PageRank (PPR) with restart probability $\alpha \in (0, 1)$, connecting each node i to the top-k nodes a walker restarting at i visits most frequently and retaining the visit probabilities as edge weights w_{ij}. Files connected via many short paths thus receive higher weight than those reached via long or cross-cluster routes. We pass w to the encoder as a scalar edge attribute that biases the attention logits of Eq. 5:

$$e_{ij}^{(\ell)} = \mathrm{LeakyReLU}\Big(a^{(\ell)\top}\big[W^{(\ell)} h_i^{(\ell)} \parallel W^{(\ell)} h_j^{(\ell)} \parallel w_{ij}\, b^{(\ell)}\big]\Big), \tag{8}$$

where $b^{(\ell)}$ is a learnable projection of the scalar edge weight, so diffusion-close neighbors receive a stronger prior in the attention distribution. The reconstruction objective remains on the original edges E, while GDC changes only the message flow within the encoder.

3.4.4 Variational Projection and Reconstruction Decoder

The final GAT layer produces file embeddings $H^{(L)}_{\texttt{file}}$; for HGAT, folder embeddings shape file representations during message passing but are not reconstructed. In all GAT-based variants, the decoder reconstructs only file–file dependency edges, so the loss in Eq. 11 remains the same. Two linear projection heads map these file embeddings to the mean (μ) and log-standard deviation ($\log\sigma$) of a Gaussian posterior for each file node i. During training, the model samples the latent embedding z_i by adding Gaussian ϵ noise scaled by σ_i:

$$z_i = \mu_i + \sigma_i \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

At inference time, we remove this randomness and use the posterior mean μ_i as the final file embedding.

A dot-product decoder reconstructs the file-level dependency graph by estimating the probability of a dependency edge between files i and j:

$$\hat{A}_{ij} = \text{sigmoid}(z_i^\top z_j). \tag{9}$$

Since observed dependencies represent only a small subset of all possible file pairs, we use negative sampling. For each set of positive dependency edges E, we sample absent file pairs S^- with $|S^-| = \rho|E|$, where ρ is the negative sampling ratio. The reconstruction loss is binary cross-entropy over positive and sampled negative pairs:

$$\mathcal{L}_{\text{recon}} = -\sum_{(i,j)\in E} \log \hat{A}_{ij} - \sum_{(i,j)\in S^-} \log\big(1 - \hat{A}_{ij}\big) \tag{10}$$

The full VGAE objective combines reconstruction with a KL-divergence regularizer that keeps the learned posterior close to a standard normal prior:

$$\mathcal{L}_{\text{VGAE}} = \mathcal{L}_{\text{recon}} + \frac{\beta}{|V_{\texttt{file}}|} \sum_{i\in V_{\texttt{file}}} \text{KL}(q(z_i) \,\|\, \mathcal{N}(0, I)), \tag{11}$$

where β is a scalar that controls the strength of the regularization. All GAT-based variants are trained with this objective and optimized with Adam [51].

3.5 Clustering and Architecture Recovery

Architectural granularity is a design choice rather than a single objectively correct answer, since systems can be organized at different levels of modular abstraction [4]. We therefore recover architectures by clustering the learned file embeddings into a flat m partition, where each source file is assigned to exactly one recovered module. The value of m can either be specified directly or selected from a candidate range $\mathcal{K}$ using the silhouette score [52], which balances intra-cluster cohesion and inter-cluster separation.

We apply agglomerative clustering with Ward linkage to all methods except N2V. For N2V, we follow the original setup of NEGAR [5] and use K-means clustering on the node2vec embeddings.

4 Experiments

We evaluate all methods studied on a collection of open-source software systems with known ground-truth (GT) labels. This section describes the benchmark systems, the experimental configurations for all techniques, and the evaluation metrics used to assess performance.

4.1 Benchmark Systems

We evaluate our approach on 11 open-source systems that are widely studied in software architecture recovery [5, 13, 53]. We use published ground-truth (GT) architectures to ensure direct comparability with prior work, as producing reliable mappings is resource-intensive and does not scale well [39]. Table 1 summarizes properties and the number of GT modules $|\mathcal{C}|$ of benchmark systems.

Table 1: Properties of the software systems. Static dependencies between files yield edges, and the average degree is computed as $\langle k\rangle = 2|E|/|V_{\texttt{file}}|$.

	System†	Version	$\lvert\mathcal{C}\rvert$	$\lvert\boldsymbol{V}_{\texttt{file}}\rvert$	$\lvert\boldsymbol{V}_{\texttt{folder}}\rvert$	|Edges|	$\langle\boldsymbol{k}\rangle$
Java	A.UML	r13713	14	766	58	3,754	9.8
	AS4	4	57	583	115	1,826	6.3
	C.Img	v1.0a2	21	329	41	1,565	9.5
	Hadoop	0.19.0	63	591	55	3,648	12.3
	JabRef	v3.7	6	1,180	127	6,126	10.4
	Lucene	r1075001	7	1,054	188	6,424	12.2
	T.Mates	v5.11	15	778	42	6,721	17.3
C/C++	Bash	4.2	13	292	11	1,010	6.9
	Chrome	svn-171054	69	18,343	999	145,492	15.9
	HDC	46ff87	11	207	87	543	5.2
	HDF	0e196f	9	153	26	535	7.0

†Datasets are available at https://github.com/sse-lnu/StructSAR.

For AS4, Hadoop, Bash, and Chromium, we use the GT labels curated by Garcia et al. [39, 3], produced through a certification process in which researchers derived an initial architectural decomposition from documentation and source code and then refined it through validation with system experts. Chromium was developed through long-term collaboration with the project's developers over roughly two years.

The GT labels for HDC were developed in collaboration with industry partners and are included in the SARIF benchmark studied by Zhang et al. [13], distributed through the SARIF replication package on GitHub[2].

The other five Java systems, ArgoUML (A.UML), Lucene, Common-Imaging (C.Img), JabRef, and TeamMates (T.Mates), were studied by Olsson et al. [53] for code-to-architecture mapping and are available through the s4rdm3x repository[3]. The GT mappings for JabRef and TeamMates were produced by system experts at the Software Architecture Erosion and Architectural Consistency Workshop (SAEroCon)[4] in 2016 and 2017. The

[2]https://github.com/anonymous2f4a9d/SARIF_FSE23
[3]https://github.com/h0bb3/s4rdm3x
[4]https://github.com/sebastianherold/SAEroConRepo

C.Img mapping was produced by its developers as a benchmark system for Olsson's study.

4.2 Classical Dependency-only Baselines

For context, we also evaluate four classic SAR methods that only use the static dependency-graph as input: ACDC [8], FCA [29], LIMBO [7], and WCA [10] variants(WCA_{UE} and WCA_{UENM}). We use the authors' original implementations for FCA, and the ARCADE framework for ACDC, LIMBO, and WCA [54]. The runner scripts are available in a replication package on GitHub[5]. ACDC and FCA automatically determine the number of clusters, whereas LIMBO and WCA stop at the ground-truth module count. For all methods, we use the same dependency graph, extracted using Depends [40], to ensure a consistent comparison.

4.3 Experimental Configuration

We use a single configuration per method across all benchmark systems, with per-system overrides only for GAT-DC (PPR top-k and layer count) and for Chrome. The selection rationale and sensitivity analysis are discussed in Section 5.3.

Node2vec. We adopt the Node2vec search space reported by Chen et al. (Table II in [5]). We ran the search on three benchmark systems also used in their study (AS4, Bash, and Hadoop) and selected one configuration that performed consistently across them: 128 embedding dimensions, walk length 30, context size 15, 100 walks per node, $p = q = 1$, five negative samples, and five training epochs. This configuration applies to all systems. For Chrome, we use the setting explicitly published by Chen et al. (one epoch and one negative sample). Unlike Chen et al., who studied each dependency type separately, we merge all types into a single edge for N2V, consistent with the common-input design across all methods; this reproduces recovery comparable to their reported results.

MetaPath2vec. We use an embedding dimension of 64, walk length 15, context size 5, and 40 walks per node. Skip-gram training uses SparseAdam with one negative sample, a learning rate of 0.05, a batch size of 32, and 20 training epochs.

Variational graph autoencoders. All GAT-based variants initialize each node type with a 128-dimensional learnable identifier embedding. For file-level nodes, this embedding is concatenated with a 16-dimensional LPE vector. Unless stated otherwise, the encoder uses 256 hidden channels, two attention heads, and is trained for 100 epochs with Adam at a learning rate of 10^{-3}. The variant-specific parameters and Chrome-specific scalability overrides are summarized in Table 2.

Chrome is substantially larger than the other benchmark systems, with about 18k file nodes. Full-batch training degraded the recovery scores of the homogeneous GAT variants (GAT and GAT-DC) in preliminary experiments, so we use link-level neighborhood sampling and reduced training settings for Chrome, as listed in Table 2. HGAT and HGAT-TD remained stable during full-batch training on the heterogeneous graph and were trained with the default capacity, including one additional GAT layer. These settings were selected for stable and competitive recovery rather than exhaustive tuning; their scalability implications are discussed in Section 5.3.

For GAT-DC, PPR diffusion expands the effective neighborhood, so larger systems use one GAT layer while smaller systems use two. The layer rule and per-system top-k values are given in Table 2 (footnotes † and ‡); the top-k sensitivity analysis is discussed in Section 5.3.

Table 2: Hyperparameters for the GAT-based variants. Values are shared within each block unless overridden below. Chrome overrides are listed only where they differ from the Default; a dash (—) means the Default applies.

	Hyperparameter	Default	Chrome
Homogen. shared	Hidden channels	256	128
	Attention heads	2	1
	β (KL)	0.1	—
	ρ (neg. ratio)	5	1
	Epochs	100	50
GAT	GAT layers	2	1
GAT-DC	GAT layers	1 or 2†	1
	PPR top-k	$\{16, 32, 64, 128\}$‡	32
	α (restart)	0.15	—
HGAT/ HGAT-TD	GAT layers	1	2
	β (KL)	0.2	—
	ρ (neg. ratio)	2	—
	Epochs	100	—

† GAT layers: 1 layer if $|V_{\texttt{file}}| > 500$ *and* $|\mathcal{C}| > 10$; 2 otherwise (see Table 1).
‡ PPR top-k: 16 for HDC/HDF, 32 for C.Img, 128 for Lucene/T.Mates, 64 for others.

We run the experiments on an Ubuntu 24.04.4 LTS server with two Intel Xeon Platinum 8468 CPUs (96 physical cores), 1 TiB RAM, and eight NVIDIA H100 80GB GPUs. Each reported run uses one H100 GPU, and threaded methods are capped at 24 CPU threads. GAT-based methods mainly benefit from the GPU, while node2vec benefits most from CPU parallelism during random-walk generation and embedding training. These settings were selected from preliminary runs to give stable runtimes without oversubscribing shared resources.

4.4 Performance Evaluation

Multiple reasonable architectural decompositions may exist for the same software system, since experts can disagree on design decisions [39]. As a result, a recovered decomposition may differ from the particular GT architecture used for evaluation. To avoid relying on any single criterion, we report a set of complementary metrics using the standard definitions and computation procedures as presented in [3].

[5] https://github.com/sse-lnu/sar-baselines

MoJoFM [55] is a primary evaluation metric in SAR. It measures how many *move* and *join* operations are needed to transform the recovered architecture A_1 into the GT architecture A_2 at the cluster level:

$$\text{MoJoFM}(A_1, A_2) = \left(1 - \frac{mno(A_1, A_2)}{\max(mno(\forall A_1, A_2))}\right) \times 100, \quad (12)$$

where *mno* is the minimum edit cost, and the denominator normalizes by its maximum possible value. However, its *join* operations carry a low cost, so a recovered partition with many small clusters can still score high if those clusters align with GT modules [56, 4].

Architecture-to-Architecture (A2A) [57] extends the edit model to also include adding, removing, and moving entities or clusters:

$$\text{A2A}(A_1, A_2) = \left(1 - \frac{mto(A_1, A_2)}{aco(A_1) + aco(A_2)}\right) \times 100, \quad (13)$$

where *mto* is the edit distance between the two architectures and *aco* is the cost of building each from scratch. A2A can return high similarity even when two architectures differ substantially in structure [3].

Cluster-to-Cluster Coverage (C2C_{cvg}) [58] counts the fraction of GT clusters that are sufficiently matched by some recovered cluster:

$$C2C_{\text{cvg}}(A_1, A_2) = \frac{|\text{simC}(A_1, A_2)|}{|A_2|} \times 100, \quad (14)$$

where $\text{simC}(A_1, A_2)$ is the set of GT clusters whose overlap with some recovered cluster exceeds a coverage threshold (th_{cvg}).

Normalized TurboMQ (TMQ_{norm}) [56] is independent of GT and measures modular quality from the dependency graph by rewarding intra-cluster cohesion and penalizing inter-cluster coupling. For k recovered clusters with intra-cluster edge weight μ_i for cluster i and inter-cluster weight ε_{ij} from cluster i to cluster j:

$$\text{TurboMQ} = \sum_{i=1}^{k} \frac{\mu_i}{\mu_i + 0.5 \times \sum_{j \neq i} (\varepsilon_{ij} + \varepsilon_{ji})}. \quad (15)$$

The TMQ_{norm} is obtained by dividing the TurboMQ by the number of recovered clusters.

In addition to the SAR-specific metrics, we also report the Adjusted Rand Index (ARI) [59], a standard clustering metric that measures agreement between the recovered and GT partitions corrected for chance, in $[-1, 1]$, with 1 indicating perfect agreement.

5 Results and Analysis

This section presents the results of our study and addresses the research questions. Our primary evaluation uses Exact-k clustering, where the number of recovered clusters is fixed to the number of GT modules. We also report *Search* results, in which the number of clusters is automatically selected within the range $\mathcal{K} = [5, 20]$, with wider per-system ranges for C.Img ($[10, 25]$), AS4 and Hadoop ($[40, 70]$), and Chrome ($[50, 75]$); these results appear in the lower block of Tables 3 and 5. Since all methods are stochastic, we run each dataset–method configuration 100 times with different seeds and report the median scores. Run-to-run variability is shown with boxplots and summarized using IQR (interquartile range).

For pairwise method comparisons, we test significance with the Mann–Whitney U test [60], which suits independent, non-Gaussian samples, and report Cliff's δ [61] as the effect size: negligible ($|\delta| < 0.147$), small ($0.147 \leq |\delta| < 0.33$), medium ($0.33 \leq |\delta| < 0.474$), and large ($|\delta| \geq 0.474$).

The metrics capture complementary aspects of recovery, so we interpret MoJoFM, A2A, C2C_{cvg}, and ARI together, with TMQ_{norm} providing a GT-independent view of structural modularity. Throughout the results, C2C_{cvg} uses a fixed coverage threshold of $th_{\text{cvg}} = 0.5$, corresponding to majority matching[56], and can therefore be more volatile near the threshold. The metrics also respond differently to cluster granularity: MoJoFM tolerates over-fragmentation, whereas ARI and C2C_{cvg} penalize it. A2A is the least discriminative of the GT-alignment metrics. We therefore rely mainly on MoJoFM and ARI to distinguish between methods.

5.1 RQ1 – Effectiveness of Dependency-only Graph Representations for Architecture Recovery

RQ1 examines how effectively software architecture can be recovered from dependency structure alone, before adding organizational or semantic information. This establishes the baseline that RQ2 builds on. We compare three learned representations on the homogeneous file–file dependency graph, N2V, GAT, and GAT-DC, together with four classic methods that rely solely on the dependency graph: ACDC, FCA, LIMBO, and variants of WCA.

All homogeneous methods use a single collapsed `depends_on` edge. In preliminary experiments, modeling dependency types separately in the GAT encoder did not improve recovery, so we revisit typed edges only in RQ2 (HGAT-TD). Table 3 reports the median GT-alignment scores for Java and C/C++ systems over 100 runs per (system, method) configuration, with the classic methods in the top block, and Table 4 reports the pairwise significance tests for the learned methods (median differences and Cliff's δ). Overall, N2V and GAT-DC achieve comparable median performance, whereas GAT is generally weaker.

No single representation dominates across all systems and metrics. Among the learned methods, GAT-DC has a slight edge over MoJoFM and ARI; N2V remains competitive on C2C_{cvg} and A2A; and GAT is generally weaker. A2A varies little across the learned methods, so MoJoFM, C2C_{cvg}, and ARI drive the discrimination between them.

ACDC and FCA decide the number of clusters themselves, while the other take it as input. ACDC achieves relatively high MoJoFM across several systems, but it can return far more clusters than a developer would consider

Table 3: Recovered-vs-GT similarity on the homogeneous file–file SDG. MoJoFM (M), A2A (A), $C2C_{cvg}$ (C), and ARI (I) are scaled by 100. In the top block, ACDC and FCA select their own cluster count while all other methods use Exact-k (the GT module count); the lower block selects the count automatically (Search). Learned-method scores are medians across 100 runs, and the best top-block score for each (system, metric) pair is highlighted.

		Java																								**C/C++**											
		A.UML				AS4				C.Img				Hadoop				JabRef				Lucene				T.Mates				Bash				Chrome				HDC				HDF			
	Method	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I
Exact-k	ACDC	72	82	3	22	74	85	41	17	82	89	34	41	42	82	9	11	76	81	1	20	68	76	0	1	48	82	4	2	59	81	10	20	68	80	2	13	57	87	12	24	47	85	6	26
	FCA	59	78	1	11	61	80	18	6	63	80	10	11	44	81	13	8	62	73	0	2	65	75	0	8	36	74	0	0	52	78	6	10	53	73	0	2	63	76	6	12	50	81	6	22
	LIMBO	59	86	21	15	66	86	37	19	59	87	19	30	48	83	11	13	62	87	17	0	50	86	14	12	63	85	20	17	58	86	31	17	51	87	7	21	55	86	9	20	48	87	22	21
	WCA_{UE}	55	86	14	24	71	85	30	16	73	86	24	29	51	84	5	21	60	85	0	−10	25	82	0	0	65	85	7	21	41	86	23	12	46	83	0	21	64	90	27	42	48	88	22	30
	WCA_{UENM}	52	87	21	24	70	85	33	17	72	86	29	26	51	84	11	20	60	85	0	−10	25	82	0	0	53	83	0	14	41	86	23	12	46	83	3	20	62	90	45	40	51	88	22	31
	N2V	75	90	43	40	72	86	40	19	80	90	43	47	52	84	11	17	72	86	17	7	73	91	57	52	66	86	7	27	63	89	31	33	74	84	30	17	71	89	18	38	55	88	33	32
	GAT	55	87	29	26	72	87	44	29	76	90	43	48	50	84	8	17	72	86	17	5	62	89	43	36	46	84	0	11	61	89	31	32	69	83	21	14	62	88	18	29	46	87	22	23
	GAT-DC	74	89	43	40	70	88	28	29	77	90	48	48	53	85	8	26	78	86	17	12	74	92	57	58	66	84	7	23	64	88	23	35	71	83	25	14	73	89	27	39	56	89	44	35
Search	N2V	77	89	30	38	72	86	40	17	79	90	50	50	51	84	11	17	75	85	7	9	73	90	33	47	67	86	8	26	58	89	60	31	72	84	36	20	72	88	17	35	58	86	18	32
	GAT	59	88	25	28	71	88	49	38	77	90	38	48	48	83	10	17	72	85	14	3	74	88	15	40	46	84	0	9	61	89	31	33	69	83	21	14	68	86	10	31	44	86	20	22
	GAT-DC	74	89	38	39	70	87	29	28	76	91	57	58	53	85	9	27	73	86	20	3	72	93	80	57	63	87	14	28	61	90	50	37	71	83	23	14	76	87	12	36	57	89	30	35

a single architecture. For example, 1,061 clusters, compared to a GT of 69, and across systems, it produces 1.7× to 30× the GT count. Because MoJoFM tolerates fine partitions, but ARI and $C2C_{cvg}$ compare partitions at a fixed granularity and drop when the count is far from the GT. This is why ACDC and FCA trail on ARI and $C2C_{cvg}$, where N2V and GAT-DC lead by about +15 to +18 ARI and +10 to +24 $C2C_{cvg}$ on average, and their lower TMQ_{norm} in Table 7 is consistent with this. The gap is not only about cluster count. LIMBO and the WCA variants use the GT count, so granularity is matched, yet the learned methods still lead them by about +12 MoJoFM and +16 ARI. HDC is the clearest exception, where the WCA variants achieve a higher ARI by a few points. Since the cluster count has such a strong effect, we treat it as a modeling choice and recover a flat partition at a chosen k, one that a developer can reason about.

The Search block broadly follows the same pattern as Exact-k: median MoJoFM stays close on most systems, so automatic k selection does not substantially change the overall ranking. The largest differences appear in $C2C_{cvg}$, whose values can shift sharply in either direction, because partitions with a different cluster count cross the fixed $C2C_{cvg}$ threshold in different ways. The Search results, therefore, mainly show $C2C_{cvg}$'s sensitivity to cluster count rather than a consistent change in recovery.

Why do GAT-DC and N2V outperform GAT? The pattern in Table 4 is consistent across the four metrics. GAT trails N2V on average, most clearly on MoJoFM (−7.4) and ARI (−6.0), while GAT-DC recovers most of that gap. Against N2V, GAT-DC shows small average gains on MoJoFM and ARI, ties on A2A, and is slightly behind on $C2C_{cvg}$. So GAT itself is not the main limitation; the limitation is applying a shallow GAT to the raw dependency graph. GAT aggregates only two-hop neighborhoods and can miss the long-range structure that separates modules, since files in the same module are often linked through shared neighbors or dependency chains rather than direct edges. Both random walks and diffusion-based rewiring expand the structural context available during representation learning, so the key factor is not only the encoder but also its ability to capture long-range dependency structure.

What limits recovery from the SDG alone? Two factors beyond the encoder bound how well methods built on dependencies alone align with the GT: the cohesion and the granularity of the GT architecture. Table 7 shows that systems with cohesive GT decompositions, roughly in the low-40s to high-50s TMQ_{norm} range (A.UML, Lucene, Chrome, AS4, and HDC), align better under these methods. Hadoop and T.Mates are the opposite case: their GT TMQ_{norm} values are much lower, around 12 to 19, so cross-module dependencies are nearly as strong as intra-module ones. These methods stay limited here, with MoJoFM below about 53 on Hadoop and 66 on T.Mates and modest ARI. Recovery is therefore bounded by how well the GT architecture is reflected in the dependency structure.

The second factor is the number of modules in the GT decomposition (granularity). Fine-grained architectures have small modules, so local context and a shallow model suffice. Coarse-grained architectures have large modules spanning many files, so the encoder needs a wider reach to pull them into a tight embedding. For GAT-DC, this reach is set by the GAT layer count and the PPR top-k; for N2V, by the walk and context lengths. This is why these settings track system size and granularity (Table 2), which we analyze in Section 5.3. Recovery from dependencies alone is thus strongest when the GT decomposition is cohesive in the dependency graph *and* the encoder's reach matches the target granularity.

Variability. Figure 2 shows the boxplots of stochastic methods across runs for systems where the variability is most pronounced. GAT-DC has the smallest average spread across the four metrics, making it the most consistent representation. This is clearest on A.UML, JabRef, Lucene, and T.Mates, especially JabRef, where GAT-DC stays tight while N2V ranges from 0 to 17 at a comparable median. $C2C_{cvg}$ varies the most because its

Table 4: Pairwise comparison of methods across 100 runs per (system, metric) cell. Each entry reports the median difference between the two methods, with Cliff's delta effect-size class in parentheses (L: large, M: medium, S: small, N: negligible). Positive values favor the first method; negative values favor the second. All differences are statistically significant (p-value < 0.05) unless marked with a †, and cells in which the second method is significantly better than the first are shaded.

	GAT − N2V				**GAT-GDC − N2V**				**GAT-GDC − GAT**			
Dataset	MoJoFM	A2A	$C2C_{cvg}$	ARI	MoJoFM	A2A	$C2C_{cvg}$	ARI	MoJoFM	A2A	$C2C_{cvg}$	ARI
A.UML	−19.9(L)	−2.7(L)	−14.3(L)	−13.9(L)	−1.3(N)†	−0.9(L)	0.0(S)	−0.7(S)†	+18.7(L)	+1.7(L)	+14.3(L)	+13.0(L)
AS4	−0.7(M)	+0.8(L)	+3.5(M)	+10.6(L)	−2.1(L)	+1.7(L)	−12.3(L)	+9.7(L)	−1.4(L)	+0.8(L)	−15.8(L)	−0.9(S)
C.Img	−4.5(L)	+0.6(S)	0.0(N)†	+0.7(S)	−3.0(L)	+0.7(M)	+4.8(M)	+0.6(N)†	+1.4(M)	+0.1(N)†	+4.8(M)	0.0(S)
Hadoop	−1.7(L)	−0.1(M)	−3.4(L)	+0.2(N)†	+1.0(L)	+1.3(L)	−3.4(L)	+9.0(L)	+2.6(L)	+1.4(L)	0.0(S)	+8.9(L)
JabRef	+0.5(N)†	−0.3(S)	0.0(S)	+0.2(S)	+5.1(L)	−0.1(S)†	0.0(S)	+4.8(L)	+4.6(L)	+0.3(S)	0.0(N)†	+4.7(L)
Lucene	−10.8(L)	−2.0(L)	−14.3(L)	−16.2(L)	+1.2(L)	+0.5(S)	0.0(S)	+6.2(L)	+12.1(L)	+2.5(L)	+14.3(L)	+22.5(L)
T.Mates	−20.8(L)	−1.9(L)	−6.7(L)	−16.1(L)	−0.1(N)†	−1.4(L)	0.0(S)	−4.2(L)	+20.8(L)	+0.5(S)	+6.7(L)	+12.0(L)
Bash	−1.2(S)	+0.8(M)	0.0(S)	−0.5(N)†	+1.6(L)	−0.7(S)	−7.7(L)	+1.9(M)	+3.2(L)	−1.5(L)	−7.7(L)	+3.0(L)
Chrome	−4.4(L)	−0.8(L)	−9.4(L)	−2.7(L)	−3.1(L)	−0.5(L)	−5.8(L)	−2.7(L)	+1.3(L)	+0.3(L)	+3.6(L)	+0.1(N)†
HDC	−9.0(L)	−0.8(L)	0.0(S)	−9.1(L)	+2.0(M)	−0.1(N)†	+9.1(S)	+1.5(S)†	+11.1(L)	+0.7(L)	+9.0(M)	+10.6(L)
HDF	−9.0(L)	−1.5(L)	−11.1(L)	−8.9(L)	+1.0(M)	+0.7(L)	+11.1(M)	+3.3(L)	+10.0(L)	+2.2(L)	+22.2(L)	+12.2(L)
Average	−7.4	−0.7	−2.5	−6.0	+0.2	+0.1	−0.4	+2.7	+7.7	+0.9	+4.7	+7.8

fixed threshold lets small file movements flip a GT module between matched and unmatched, so much of the spread reflects threshold crossings rather than real differences in recovery. Under Search, automatic k selection adds a second source of randomness and affects the methods unevenly: GAT-DC stays stable in ARI (average IQR 2.2 to 2.4), whereas N2V widens from 4.0 to 5.7. This increase is concentrated in coarse-grained systems, where the silhouette score shows no sharp peak and the selected k varies across runs: the N2V ARI IQR rises from 8.7 to 19.1 on JabRef and from 4.7 to 18.2 on Lucene. Here, the instability for N2V likely arises from both embedding noise and uncertainty in the number of clusters. The spread is also small relative to the gap over the baselines: where N2V or GAT-DC lead ACDC on ARI, even their low runs stay ahead, except on JabRef where ACDC leads outright.

Answer to RQ1. Representations built on dependencies alone provide a useful but limited basis for SAR. N2V and GAT-DC achieve comparable performance, with GAT-DC slightly stronger on MoJoFM and ARI and the most stable, while GAT alone is generally weaker. This suggests that two-hop message passing on the raw dependency graph is often too local, and that both random-walk context and diffusion-based rewiring help expose longer-range dependency structure. However, recovery remains bounded by how clearly the GT architecture is reflected in the dependency graph, and by how well the model's use of dependency context matches the target granularity. The lead of GAT-DC and N2V over deterministic baselines on fragmentation-sensitive metrics (ARI and $C2C_{cvg}$) holds across runs.

5.2 RQ2 – Impact of Project Organization Signal on Architecture Recovery

In RQ2, we examine whether folder organization adds architectural information beyond the cohesion and coupling signals in the SDG. We model this using folder nodes and `folder-file` containment edges, and compare M2V, HGAT, and HGAT-TD against the homogeneous baselines N2V and GAT-DC from RQ1 (Table 5). We ask whether combining dependencies with folder co-membership improves alignment with the human-defined GT mapping, and when this signal is most useful. We also report GAER [4] as a reference point, since it additionally incorporates code semantics[6].

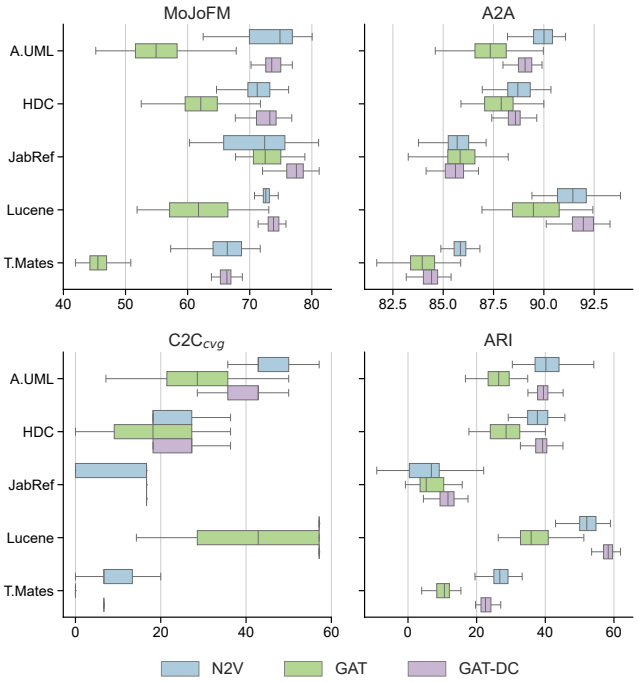


Figure 2: Score distributions over 100 runs for A.UML, HDC, JabRef, Lucene, and T.Mates.

Across both Java and C/C++ systems, the heteroge-

[6] For datasets shared in the GAER paper [4], we report the published numbers. For A.UML, C.Img, and Lucene, we ran GAER using the publicly released implementation [62].

Table 5: Recovered-vs-GT similarity on the heterogeneous graph (file–file SDG plus folder nodes and containment edges), median over 100 runs. Metrics MoJoFM (M), A2A (A), C2C$_{cvg}$ (C), and ARI (I) are scaled by 100. The upper block fixes the cluster count to the GT module count (Exact-k); the lower block selects it automatically (Search). The strongest dependency-only methods (N2V and GAT-DC) from RQ1 are repeated in the top block. GAER is shown in italics as a reference point, since it additionally uses source-code semantics. The best Exact-k result per (system, metric) is highlighted.

		Java																						**C/C++**											
		A.UML				AS4				C.lmg				Hadoop				JabRef				Lucene				T.Mates				Bash				Chrome				HDC				HDF			
	Method	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I
	GAER	*94*	*97*	*86*	*83*	*80*	*88*	*42*	*27*	*93*	*91*	*48*	*49*	*69*	*86*	*30*	*28*	*91*	*90*	*33*	*34*	*92*	*94*	*68*	*58*	*94*	*90*	*47*	*51*	*91*	*93*	*64*	*58*	*91*	*87*	*33*	*30*	*89*	*95*	*73*	*71*	*90*	*97*	*78*	*76*
Exact-k	N2V	75	90	43	40	72	86	40	19	80	90	43	47	52	84	11	17	72	86	17	7	73	91	57	52	66	86	7	27	63	89	31	33	74	84	30	17	71	89	18	38	55	88	33	32
	GAT-DC	74	89	43	40	70	88	28	29	77	90	48	48	53	85	8	26	78	86	17	12	74	92	57	58	66	84	7	23	64	88	23	35	71	83	25	14	73	89	27	39	56	89	44	35
	M2V	89	93	57	60	75	88	40	26	87	92	50	55	62	87	21	33	69	86	17	1	74	91	43	49	81	89	33	41	78	95	50	66	74	88	32	37	72	89	27	41	67	92	56	57
	HGAT	91	95	64	68	77	88	44	27	84	92	52	52	61	85	21	24	75	85	17	11	71	90	43	45	80	91	47	50	93	95	69	71	81	87	23	33	74	89	36	42	73	93	67	65
	HGAT-TD	89	94	57	62	78	88	44	26	82	91	48	50	64	86	23	29	75	85	17	10	71	90	43	45	79	90	40	46	93	96	77	75	83	87	26	34	74	89	36	42	74	93	67	60
Search	M2V	90	91	40	48	77	87	41	25	89	92	42	53	63	87	23	33	79	82	0	10	90	88	20	47	85	89	35	42	79	95	40	66	73	89	37	40	82	86	20	35	78	93	35	68
	HGAT	93	93	50	62	80	88	44	25	86	92	46	51	61	86	27	33	80	82	0	11	87	88	20	45	85	91	45	49	92	94	60	68	82	87	24	33	82	86	20	34	92	94	40	78
	HGAT-TD	92	93	50	59	80	88	44	25	85	91	46	50	65	87	27	36	80	82	0	11	87	88	20	46	86	91	40	48	92	96	67	71	84	87	25	33	82	85	20	33	88	92	35	71

neous methods outperform N2V and GAT-DC on most (system, metric) cells, with the largest gains where the dependency signal alone fell short. On Hadoop, MoJoFM rises from at most 53 (GAT-DC) to 62 (M2V) and 64 (HGAT-TD), and ARI from 26 to 33. On T.Mates, MoJoFM increases from 66 to 81 (M2V) and 80 (HGAT), and C2C$_{cvg}$ from 7 to 33 (M2V) and 47 (HGAT). The C/C++ results match: Bash improves from 63–64 to 93 under HGAT and HGAT-TD, and HDF from 55–56 to the low-70s. Folder containment yields the largest gains on systems with low TMQ$_{norm}$ on GT (Hadoop, T.Mates) or sparse dependency graphs relative to module count (Bash, HDF), suggesting that folder organization carries architectural information not present in the SDG alone.

Under Search, the heterogeneous methods behave differently from the homogeneous ones. When silhouette selection chooses more clusters than the GT, GT modules are split into finer parts, increasing MoJoFM and ARI but lowering C2C$_{cvg}$. HDF is the clearest example: although the GT has nine modules, silhouette selection always chooses 20 clusters, so HGAT's MoJoFM rises from 73 to 92, ARI also improves, and C2C$_{cvg}$ drops sharply because GT modules are distributed across multiple recovered clusters. HDC and Lucene show the same pattern. These gains are therefore real but metric-dependent, and remain meaningful because the cluster ranges are small: the finer partitions still provide a useful architectural view and can be merged if a coarser interpretation is preferred.

What does folder structure add, and where does it stop helping? Folder nodes shift the learning signal away from pure dependency graph toward the organizational decisions encoded in the directory tree. The improvements in Table 5 indicate that, on most systems, folder placement aligns reasonably well with the GT architecture, reflecting that incorporating folder co-membership better aligns with the human-defined mappings. Systems where the GT decomposition has very low cohesion in the dependency graph, or has many fine-grained modules relative to graph density, gain the most because the organizational signal supplies boundary information that is lacking in the dependencies.

However, folder location is not always enough. JabRef is the clearest counter-example: its source is split across 127 folders for 1,180 files, so any single `folder-file` link captures only a narrow slice of organizational context. All three heterogeneous methods stay below the GAT-DC baseline built on dependencies alone (78 MoJoFM): M2V at 69, HGAT and HGAT-TD at 75. GAER, which uses information beyond graph structure, reaches 91 on JabRef, but its advantage is neither uniform nor metric-independent. HGAT and HGAT-TD beat GAER on all four metrics for Bash, and M2V or HGAT variants match or beat GAER on A2A, C2C$_{cvg}$, or ARI for AS4, C.lmg, Hadoop, T.Mates, and Chrome. GAER leads on all four metrics for A.UML, JabRef, HDC, and HDF, with its clearest edge on MoJoFM. Structural and organizational signals stay competitive in partition-level agreement and coverage, even where GAER is strongest on MoJoFM.

How do HGAT and M2V use folder structure? To assess whether HGAT's advantage over M2V is consistent across runs, Table 6 (right block) reports pairwise comparisons over 100 runs using median score differences and Cliff's δ, following the same protocol as RQ1 5.1.

Table 6: Pairwise comparison across 100 runs per (system, metric) cell: HGAT against HGAT-TD (left block) and HGAT against M2V (right block). Each entry reports the median difference with Cliff's delta effect sizes (L: large, M: medium, S: small, N: negligible). Positive values favor the first method. Differences are significant ($p < 0.05$) unless marked †, and cells where the second method significantly wins are shaded.

	HGAT − HGAT-TD				**HGAT − M2V**			
Dataset	MoJoFM	A2A	C2C$_{cvg}$	ARI	MoJoFM	A2A	C2C$_{cvg}$	ARI
A.UML	$+1.1(L)$	$+1.1(L)$	$+7.1(S)$	$+5.9(L)$	$+1.7(L)$	$+1.6(L)$	$+7.1(M)$	$+7.7(L)$
AS4	$-0.2(N)^{\dagger}$	$+0.3(M)$	$+0.0(S)$	$+0.8(L)$	$+2.1(L)$	$+0.7(L)$	$+3.5(L)$	$+1.1(L)$
C.lmg	$+1.9(M)$	$+0.5(L)$	$+4.8(S)$	$+2.1(L)$	$-2.9(L)$	$-0.3(M)$	$+2.4(N)^{\dagger}$	$-2.3(L)$
Hadoop	$-3.6(L)$	$-1.1(L)$	$-1.9(L)$	$-5.3(L)$	$-1.4(L)$	$-2.6(L)$	$-0.3(M)$	$-9.5(L)$
JabRef	$-0.2(N)^{\dagger}$	$-0.3(N)^{\dagger}$	$+0.0(L)$	$+1.1(N)^{\dagger}$	$+6.3(L)$	$-0.7(L)$	$+0.0(N)^{\dagger}$	$+10.5(L)$
Lucene	$-0.1(N)^{\dagger}$	$-0.3(S)$	$+0.0(N)^{\dagger}$	$+0.6(N)^{\dagger}$	$-2.7(L)$	$-0.6(M)$	$+0.0(S)$	$-3.5(L)$
T.Mates	$+0.9(S)$	$+0.4(L)$	$+6.7(S)$	$+4.0(L)$	$-0.9(M)$	$+1.8(L)$	$+13.3(L)$	$+8.6(L)$
Bash	$+0.0(N)^{\dagger}$	$-1.9(L)$	$-7.7(L)$	$-4.5(L)$	$+14.4(L)$	$-0.3(S)^{\dagger}$	$+19.2(L)$	$+5.0(L)$
Chrome	$-2.3(L)$	$-0.2(M)$	$-2.9(L)$	$-0.9(M)$	$+6.8(L)$	$-0.9(L)$	$-8.7(L)$	$-3.4(L)$
HDC	$+0.2(N)^{\dagger}$	$-0.1(N)^{\dagger}$	$+0.0(S)$	$+0.6(N)^{\dagger}$	$+2.5(S)$	$-0.2(N)^{\dagger}$	$+9.1(M)$	$+1.3(S)^{\dagger}$
HDF	$-1.7(S)$	$-0.3(M)$	$+0.0(S)$	$+5.1(L)$	$+5.9(L)$	$+0.8(S)$	$+11.1(M)$	$+8.3(L)$
Average	-0.4	-0.2	$+0.6$	$+0.9$	$+2.9$	-0.1	$+5.2$	$+2.2$

HGAT and M2V use the folder signal differently. For M2V, folder structure is the learning objective: meta-path walks pull files that share a folder together through the Skip-gram loss, so the model learns folder co-membership directly, but only from the immediate parent folder. For HGAT, folder structure is a supporting context: folder nodes join relation-specific message passing and shape the file embeddings, while the loss reconstructs file-to-file dependency edges, so the encoder learns relation-specific weights directly without a fixed schema. This pattern is visible in the per-system results. HGAT is significantly better on systems with deeper or more fragmented folder hierarchies (A.UML, AS4, JabRef, Bash, HDF, HDC), where folder nodes in the message passing help the encoder use folder context alongside dependencies. M2V is significantly better on Hadoop, C.Img, and Lucene, where each file's parent folder closely matches its GT module, allowing the meta-path walker to connect co-located files directly.

On average across the eleven systems, HGAT exceeds M2V by +2.9 MoJoFM, +5.2 $C2C_{cvg}$, and +2.2 ARI, with A2A tied. The two methods are therefore not interchangeable: HGAT is the more general choice. M2V is also more sensitive to learning rate and embedding dimension than HGAT (Section 5.3). Chrome is the one configuration exception: HGAT uses two layers rather than one because the system is large enough (18,000 files, nearly 1,000 folders) that file context lies more than one hop away. This reflects the same reach-versus-size principle observed for GAT-DC in RQ1, where larger systems need wider context, and here the encoder supplies it via an extra layer rather than via PPR rewiring.

Does modeling dependency types separately help? We also test whether modeling dependency types separately adds value beyond the folder signal. HGAT-TD keeps dependency types such as calls and imports as separate relations, but changes little overall and provides only a modest, system-dependent benefit. It sometimes improves MoJoFM, but its effect on $C2C_{cvg}$ and ARI is mixed and often slightly negative, likely because splitting dependencies by type makes each relation sparser and can fragment clusters. The clearest exceptions are Hadoop and Bash, where separating dependency types appears to reduce cross-module noise and improve several metrics. Across systems, the average differences are small, so dependency typing appears to be a system-specific refinement rather than a reliable improvement.

Variability. Figure 3 compares the run-to-run distributions of MoJoFM and ARI across heterogeneous methods on selected systems. Bash is the most stable: HGAT and HGAT-TD are nearly flat while M2V spreads a few points wider. On HDC and HDF, all three are comparably variable, with HGAT variants the widest on MoJoFM, so no method is clearly consistent there, yet all three stay well above homogeneous methods. HGAT-TD tracks HGAT throughout the spread, so modeling the dependency types separately does not change stability. JabRef is an exception: M2V's ARI remains near zero, so its low spread reflects consistent failure, and all three heterogeneous methods fall below GAT-DC. Overall, variability does not weaken the main RQ2 result: across all other systems, the heterogeneous methods maintain medians and distributions above those of N2V and GAT-DC. Their spread also stays similar or decreases under Search, with the average IQR over MoJoFM and ARI dropping from 3.3 to 2.0 for HGAT, nearly unchanged for M2V (3.5 to 3.2), and flat for HGAT-TD (3.1).

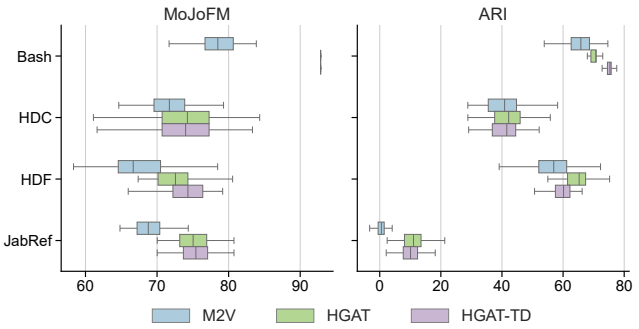


Figure 3: Distribution of scores over 100 runs for MoJoFM (left) and ARI (right) for Bash, HDC, HDF, and JabRef.

5.2.1 Internal Modularity

Table 7 reports TMQ_{norm} for the recovered decompositions alongside the GT decomposition. Since TMQ_{norm} measures dependency-based modularity, higher values mean stronger intra-cluster cohesion and weaker inter-cluster coupling. The dependency-only methods, especially N2V, reach the highest TMQ_{norm} on most systems, so random-walk embeddings capture dependency cohesion particularly strongly; the GAT-based homogeneous variants follow a similar pattern.

Table 7: Normalized TurboMQ scores. The GT row reports the score obtained by applying the published GT clustering to the dependency graph; the remaining rows report recovered scores per method. The upper block lists the homogeneous methods, and the lower block the heterogeneous methods and GAER. The best recovered score per dataset is highlighted.

	Java							C/C++			
Method	A.UML	AS4	C.Img	Hadoop	JabRef	Lucene	T.Mates	Bash	Chrome	HDC	HDF
GT	44	43	38	12	30	58	19	36	47	43	35
ACDC	20	36	39	24	13	28	15	34	31	65	41
FCA	18	29	22	20	18	22	11	32	27	57	23
LIMBO	37	43	31	31	43	59	19	48	12	48	44
WCA_{UE}	39	39	41	22	67	14	14	54	6	59	41
WCA_{UENM}	43	43	42	25	67	14	18	54	7	50	59
N2V	77	52	60	39	85	85	33	73	67	84	72
GAT	67	52	50	34	65	90	27	74	51	86	61
GAT-DC	71	36	44	22	65	71	25	53	58	71	68
GAER	*59*	*37*	*53*	*30*	*64*	*72*	*13*	*43*	*65*	*67*	*49*
M2V	65	37	58	22	85	72	13	33	53	67	41
HGAT	65	39	63	29	70	73	12	37	50	69	48
HGAT-TD	66	39	61	30	73	74	13	39	50	68	54

The heterogeneous methods and GAER generally obtain lower TMQ_{norm} than the strongest homogeneous

methods. This is not a weaker recovery but a shift in objective: TurboMQ rewards decompositions that minimize cross-cluster *dependency* edges, whereas M2V and HGAT also group files by folder containment, and GAER adds semantic signals from identifiers and naming. These signals move the recovered decomposition away from pure dependency cohesion, yet the improved GT alignment in Table 5 suggests the GT architecture is not based on dependencies alone. It also reflects how developers organize and name the system, and that organization may inform how experts draw the GT, so directory layout and the GT are not fully independent. Methods built on dependencies thus favor structurally cohesive partitions, whereas organization- and semantically aware methods better capture developer-defined boundaries, consistent with the view that no single decomposition is objectively correct [4, 3].

Answer to RQ2. Augmenting the dependency graph with folder nodes and containment edges improves GT alignment on most systems, most clearly where the dependency signal alone is weak, so project organization carries architectural information that dependencies do not capture. HGAT is the more robust choice, especially when the folder hierarchy is deep or fragmented, while M2V does better when a file's anchor folder already aligns with its GT module. Modeling dependency types separately (HGAT-TD) is a system-dependent refinement rather than a uniform gain, yielding only about a 1-point gain on HGAT on average. Overall, folder structure remains a partial signal: it complements dependencies but is insufficient when the layout diverges from the architecture, and its gains come with lower $\mathrm{TMQ}_{\mathrm{norm}}$, as recovery shifts from dependency modularity toward developer-defined boundaries.

5.3 RQ3 – Practical Considerations for Adoption

RQ3 examines two practical concerns: runtime as systems scale and per-system hyperparameter tuning. We report runtime first, then analyze sensitivity to the method-specific hyperparameters in Table 2.

5.3.1 Scalability

Table 8 reports end-to-end runtime under the Search setting, excluding dependency extraction since it is shared by all methods and measured after extraction with Depends [40]. All methods finish in under 60 seconds on every system except Chrome, where runtimes range from 57 seconds (M2V) to 289 seconds (N2V). M2V and HGAT are the fastest: they avoid both N2V's 100 random walks and GAT-DC's denser rewired graph, where keeping up to k neighbors per node typically gives the GAT encoder more edges than the raw dependency graph. HGAT-TD is slower than HGAT, since splitting the dependency edges by type raises the relation count and the per-layer message passing cost. N2V runs on the CPU via the Gensim library [63] to stay comparable to Chen et al. [5], while all other methods use a GPU. Even on Chrome, every method completes in under five minutes, so the studied representation learning methods are not the scalability bottleneck for SAR adoption.

Table 8: Average runtime comparison across the benchmark systems under the Search setting where k is searched over a predefined search range. The values are reported in seconds and averaged over the three runs.

System	N2V	GAT	GAT-DC	M2V	HGAT	HGAT-TD
A.UML	20.21	3.28	3.40	2.55	2.35	4.76
AS4	16.63	2.26	2.22	1.91	1.88	4.37
C.Img	8.17	1.44	1.50	1.00	1.21	3.75
Hadoop	18.56	3.25	3.18	2.15	2.40	4.96
JabRef	25.59	5.08	5.39	3.94	3.63	6.17
Lucene	29.17	5.18	5.59	3.45	3.62	5.93
T.Mates	22.06	4.92	5.32	2.77	3.24	5.70
Bash	5.69	1.13	1.21	0.90	0.94	2.95
Chrome	289.46	87.55	100.67	56.69	69.17	89.29
HDC	4.14	0.92	0.98	0.55	0.75	3.07
HDF	2.48	0.85	0.92	0.44	0.71	2.71

5.3.2 Hyperparameter Sensitivity

This section characterizes how the main hyperparameters affect recovery. We report it as a sensitivity analysis rather than an automatic selection rule: the per-system values in Table 2 were fixed during development, and the analysis below explains how they relate to system properties such as size and GT granularity.

The PPR top-k parameter controls how many diffused neighbors each node retains after rewiring: larger values broaden dependency context, while smaller values keep it local. Because the useful context depends on each system's dependency structure, we sweep $k \in 16, 32, 64, 128$ per system. Table 9 reports median scores over 100 runs for each (system, top-k) setting. Since MoJoFM favors smaller k and ARI favors larger k, we interpret them jointly rather than optimizing either metric alone.

Across systems, the best top-k tracks GT granularity. Finer-grained architectures split at a smaller top-k because each module has fewer files and needs less diffuse context, whereas coarse-grained systems require wider context. The two smallest and sparsest systems, HDC and HDF (fewer than 250 files), settle at top-$k = 16$. Lucene, with seven large modules over 1,054 files, and T.Mates, with the highest average degree (17.3), need the widest context and use top-$k = 128$. System size and average degree are secondary: C.Img and Bash are close in size (329 versus 292 files) but prefer top-$k = 32$ and 64, tracking module counts (21 versus 13) rather than size. For Chrome, $k > 32$ was infeasible, since the diffusion matrix grows too dense at roughly 18k nodes.

AS4 and Hadoop show why we read metrics jointly. Both have fine-grained GTs (57 and 63 modules) with MoJoFM nearly flat across top-k (within one point), so

Table 9: Median GT-alignment scores for GAT-DC across four top-k values, over 100 runs per (system, top-k) configuration. MoJoFM (M), A2A (A), $C2C_{cvg}$ (C), and ARI (I) are scaled by 100. The best value per metric is highlighted.

	top-k=16				top-k=32				top-k=64				top-k=128			
System	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I
A.UML	70	89	43	34	73	89	43	37	74	89	43	40	74	89	36	41
AS4	71	86	32	15	70	85	25	17	70	88	28	29	68	87	26	27
C.Img	79	90	43	44	77	90	48	48	73	89	38	43	71	89	33	46
Hadoop	52	83	6	15	53	85	10	22	53	85	8	26	52	84	6	24
JabRef	75	85	17	9	77	85	17	10	78	86	17	12	76	85	17	7
Lucene	65	89	43	35	70	89	57	40	71	89	57	40	74	92	57	58
T.Mates	63	85	13	20	55	84	13	14	61	83	7	15	66	84	7	23
Bash	62	86	23	28	64	87	23	33	64	88	23	35	60	87	19	32
Chrome	69	83	22	14	71	83	25	14	—	—	—	—	—	—	—	—
HDC	73	89	27	39	70	88	18	40	61	86	9	31	62	87	27	36
HDF	56	89	44	35	55	88	33	34	55	88	44	33	47	87	22	22

MoJoFM alone does not distinguish between the settings. ARI breaks the tie: on AS4 it rises from 15 at top-k = 16 to 29 at top-k = 64, and on Hadoop it peaks at top-k = 64 (26, versus 15 at top-k = 16). We set top-k = 64 for both, as the gain in partition-level agreement outweighs the slight drop in MoJoFM. Layer count follows the same logic: GAT-DC uses one GAT layer on large, fine-grained systems, where PPR already widens each node's reach, and two layers on smaller or coarser systems.

Tuning M2V. M2V was harder to tune than the other methods and was sensitive to both the learning rate and the embedding dimension. A learning rate of 0.05 and an embedding dimension of 64 produced stable training across all 11 systems. We attribute this sensitivity to the strong folder-anchor signal from the minimal meta-path schema: files that share a folder often co-occur in short walks, which can make the embeddings folder-dominated. The schema also matters; as noted in Section 5.2, longer schemas required hand-designed traversal orders and reverse-edge handling, and underperformed compared to the minimal `file-folder-file-file` path. HGAT-TD adds no tuning beyond HGAT; it reuses the HGAT configuration and only splits dependency edges by type.

Answer to RQ3. Runtime is not a bottleneck: every method finishes in under a minute except on Chrome (∼18k nodes), which takes one to five minutes. Hyperparameter sensitivity is limited and predictable. For GAT-DC, the best top-k tracks target granularity: 16 for the smallest and sparsest systems, 32 to 64 for mid-sized ones, and 128 for the densest or most coarse-grained systems (Lucene and T.Mates). Large swings appear only when top-k is far from a system's scale. M2V is more sensitive than the GAT-based variants to learning rate and embedding dimension, but trains stably across all 11 systems with learning rate 0.05, embedding dimension 64, and the minimal `file-folder-file-file` meta-path.

6 Threats to Validity

External validity. Our evaluation covers eleven open-source systems in Java, C, and C++ with available ground-truth module mappings. These systems sample only part of the real-world variation in size, domain, and architectural style, so the findings may not generalize to all systems. The benchmark also excludes Python and other dynamically typed languages, in which dependency extraction relies less on static imports and more on runtime behavior. We expect the relative ordering of the embedding methods to hold, since they operate on the dependency graph rather than on source syntax, but confirming this needs systems and GT in those languages. We likewise do not test code written largely by AI coding assistants, a setting that is becoming common and may produce different dependency structures from human-generated code.

Internal Validity. We recover architecture at the file-level granularity, which gives a coarser view than class- or method-level analysis. We chose files because the published ground-truth mappings are defined over files and because file-level dependency graphs are cheaper to build and more stable across systems. The cost is that fine-grained relations within a file are not visible to the model, and a file that spans more than one architectural concern cannot be split. A further limitation is that dependencies are recovered using a static analysis tool, which may miss edges arising from dynamic features or incomplete parsing, and may introduce non-existent ones. This affects all dependency-based recovery methods, but an inaccurate input graph can under- or over-represent the true coupling between files.

The quality of the ground-truth decompositions is another threat. These mappings are created by people and can involve judgment calls, especially for borderline artifacts, so different experts may not always agree. For some systems (e.g., JabRef, TeamMates, and CommonImaging), the mappings were provided by system experts; for HDC and HDF, they were produced by industry partners; and for AS4, Hadoop, Bash, and Chrome, the mappings were reconstructed from documentation and implementation details as part of research projects. These datasets are widely used in SAR research, but small errors or subjective choices in the GT can affect the reported alignment scores.

The statistical analysis also introduces threats. We perform pairwise comparisons across systems, methods, and metrics, which increases the chance of observing some significant results by chance. In addition, each comparison uses 100 stochastic runs, so tests can detect small distributional differences that may not be practically meaningful. We therefore interpret p-values alongside median differences and Cliff's delta effect sizes, and treat metrics with uniformly small absolute differences, such as A2A, as secondary evidence rather than as decisive

method rankings.

Ethical considerations. A recovered architecture describes how a system is structured, not how well its developers perform. We therefore caution against using these decompositions to evaluate individual engineers or to drive productivity and KPI dashboards, since module boundaries reflect design and dependency history rather than developer effort. The input graphs are built from file paths and static dependencies, which can expose identifying details such as author names or internal directory layouts. We use only publicly available GT benchmarks and release no private code or data. Our implementation and processed graphs are available in the linked repository 1 to support replication.

7 Conclusion

We studied software architecture recovery from static dependency graphs by learning self-supervised file embeddings and clustering them into architectural modules, across eleven benchmark systems. Recovery from dependencies alone is useful but bounded, and the bound is the model's reach rather than the encoder: shallow message passing on the raw graph stays too local, while random-walk context and diffusion rewiring expose the longer-range structure that separates modules. Among these methods, GAT-DC and N2V are comparable, with GAT-DC slightly ahead and the most stable.

When dependency cohesion is weak, project organization supplies architectural information that dependencies do not capture, and it helps most exactly where recovery from dependencies alone struggles, with HGAT the most robust choice among the heterogeneous methods. The gain comes with a trade-off: these methods align better with the GT by moving away from pure dependency cohesion, so their TMQ_{norm} falls below the strongest methods built on dependencies. Maximizing dependency modularity and recovering the developer-intended architecture are therefore related but not identical goals.

The methods are also scalable. Every method finishes in under a minute except on Chrome (about 18k nodes, one to five minutes), so representation learning is not the bottleneck for SAR adoption, and tuning sensitivity is limited and follows predictable patterns.

In future work, we plan to model dependency types jointly rather than in isolation. HGAT-TD applies a separate GAT layer per type but does not learn across them. Treating typed dependencies as meta-paths and attending over them would let the model weigh relation types jointly, and a heterogeneous decoder that reconstructs typed and organizational edges would let relational detail shape the embeddings on both sides of the autoencoder rather than only the encoder.

References

1. Clements P, Garlan D, Little R, Nord R, and Stafford J. Documenting software architectures: views and beyond. *25th International Conference on Software Engineering, 2003. Proceedings*. IEEE. 2003 :740–1
2. Parnas DL. On the criteria to be used in decomposing systems into modules. Communications of the ACM 1972; 15:1053–8
3. Lutellier T, Chollak D, Garcia J, Tan L, Rayside D, Medvidovic N, and Kroeger R. Comparing software architecture recovery techniques using accurate dependencies. *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. Vol. 2. IEEE. 2015 :69–78
4. Jabeen R, Ericsson M, Nordqvist J, and Wingkvist A. GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery. Proceedings of the ACM on Software Engineering 2026 Jul; 3. DOI: 10.1145/3808187
5. Chen J, Wang Z, Jiang Y, Pang J, Zhang T, Pan M, and Sun J. Negar: network embedding guided architecture recovery for software systems. *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE. IEEE, 2022 :367–76
6. Naseem R, Deris MM, Maqbool O, and Shahzad S. Euclidean space based hierarchical clusterers combinations: an application to software clustering. Cluster Computing 2019; 22:7287–311
7. Andritsos P, Tsaparas P, Miller RJ, and Sevcik KC. LIMBO: Scalable clustering of categorical data. *International Conference on Extending Database Technology*. Springer. 2004 :123–46
8. Tzerpos V and Holt RC. Accd: an algorithm for comprehension-driven clustering. *Proceedings Seventh Working Conference on Reverse Engineering*. IEEE. 2000 :258–67
9. Mitchell BS and Mancoridis S. On the automatic modularization of software systems using the bunch tool. IEEE Transactions on Software Engineering 2006; 32:193–208
10. Maqbool O and Babri HA. The weighted combined algorithm: A linkage algorithm for software clustering. *Eighth European Conference on Software Maintenance and Reengineering, 2004. CSMR 2004. Proceedings*. IEEE. 2004 :15–24
11. Garcia J, Popescu D, Mattmann C, Medvidovic N, and Cai Y. Enhancing architectural recovery using concerns. *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*. IEEE. 2011 :552–5
12. Corazza A, Di Martino S, Maggio V, and Scanniello G. Investigating the use of lexical information for software system clustering. *2011 15th European Conference on Software Maintenance and Reengineering*. IEEE. 2011 :35–44

13. Zhang Y, Xu Z, Liu C, Chen H, Sun J, Qiu D, and Liu Y. Software architecture recovery with information fusion. *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2023 :1535–47
14. Papachristou M. Software clusterings with vector semantics and the call graph. *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2019 :1184–6
15. Rukmono SA, Ochoa L, and Chaudron M. Deductive software architecture recovery via chain-of-thought prompting. *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. 2024 :92–6
16. Veličković P, Cucurull G, Casanova A, Romero A, Lio P, and Bengio Y. Graph attention networks. arXiv preprint arXiv:1710.10903 2017
17. Kipf TN and Welling M. Variational graph auto-encoders. arXiv preprint arXiv:1611.07308 2016
18. Mancoridis S, Mitchell BS, Chen Y, and Gansner ER. Bunch: A clustering tool for the recovery and maintenance of software system structures. *Proceedings IEEE International Conference on Software Maintenance-1999 (ICSM'99).'Software Maintenance for Business Change'(Cat. No. 99CB36360)*. IEEE. 1999 :50–9
19. Parsa S and Bushehrian O. A new encoding scheme and a framework to investigate genetic clustering algorithms. Journal of Research and Practice in Information Technology 2005; 37:127–43
20. Anquetil N and Lethbridge TC. Recovering software architecture from the names of source files. Journal of Software Maintenance: Research and Practice 1999; 11:201–21
21. Risi M, Scanniello G, and Tortora G. Using fold-in and fold-out in the architecture recovery of software systems. Formal Aspects of Computing 2012; 24:307–30
22. Kargar M, Isazadeh A, and Izadkhah H. Semantic-based software clustering using hill climbing. *2017 international symposium on computer science and software engineering conference (CSSE)*. IEEE. 2017 :55–60
23. Puchala SPR, Chhabra JK, and Rathee A. Ensemble clustering based approach for software architecture recovery. International Journal of Information Technology 2022; 14:2013–9
24. Sadat Jalali N, Izadkhah H, and Lotfi S. Multi-objective search-based software modularization: Structural and non-structural features. Soft Computing-A Fusion of Foundations, Methodologies & Applications 2019; 23
25. Rathee A and Chhabra JK. Software remodularization by estimating structural and conceptual relations among classes and using hierarchical clustering. *Advanced Informatics for Computing Research: First International Conference, ICAICR 2017, Jalandhar, India, March 17–18, 2017, Revised Selected Papers*. Springer. 2017 :94–106
26. Harman M and Yao X. Software module clustering as a multi-objective search problem. IEEE Transactions on Software Engineering 2010; 37:264–82
27. Naseem R, Maqbool O, and Muhammad S. Cooperative clustering for software modularization. Journal of Systems and Software 2013; 86:2045–62
28. Altınışık M, Sözer H, and Gürsun G. Software architecture recovery from multiple dependency models. *Proceedings of the 39th ACM/SIGAPP symposium on applied computing*. 2024 :1185–92
29. Teymourian N, Izadkhah H, and Isazadeh A. A fast clustering algorithm for modularization of large-scale software systems. IEEE Transactions on Software Engineering 2020; 48:1451–62
30. Mohammadi S and Izadkhah H. A new algorithm for software clustering considering the knowledge of dependency between artifacts in the source code. Information and Software Technology 2019; 105:252–6
31. Pourasghar B, Izadkhah H, Isazadeh A, and Lotfi S. A graph-based clustering algorithm for software systems modularization. Information and Software Technology 2021; 133:106469
32. Misra J, Annervaz K, Kaulgud V, Sengupta S, and Titus G. Software clustering: Unifying syntactic and semantic features. *2012 19th Working Conference on Reverse Engineering*. IEEE. 2012 :113–22
33. Zhao W, Jin W, Zhang Y, Fan M, Wang H, Li L, Liu Y, and Liu T. Software Architecture Recovery Augmented with Semantics. IEEE Transactions on Software Engineering 2025
34. Jabeen R, Ericsson M, Nordqvist J, and Wingkvist A. Graph Convolution Networks for Mapping Source Code Entities to Architectural Modules. *22nd IEEE International Conference on Software Architecture (ICSA 2025), Odense, Denmark, March 31-April 4, 2025*. IEEE. 2025
35. Bedi P and Sharma C. Community detection in social networks. Wiley interdisciplinary reviews: Data mining and knowledge discovery 2016; 6:115–35
36. Turner E. Graph auto-encoders for financial clustering. arXiv preprint arXiv:2111.13519 2021
37. Afzali A, Hosseini H, Mirzai M, and Amini A. Clustering time series data with gaussian mixture embeddings in a graph autoencoder framework. *International Workshop on AI for Transportation*. Springer. 2025 :82–107
38. Stavropoulou I, Grigoriou M, and Kontogiannis K. Case study on which relations to use for clustering-based software architecture recovery. Empirical Software Engineering 2017; 22:1717–62
39. Garcia J, Krka I, Mattmann C, and Medvidovic N. Obtaining ground-truth software architectures. *2013 35th International Conference on Software Engineering (ICSE)*. IEEE. 2013 :901–10
40. Nguyen HV. Depends: Multilingual Dependency Analysis Tool. `https://github.com/multilang-depends/depends`. Accessed: 2025-08-17. 2023
41. Grover A and Leskovec J. node2vec: Scalable feature learning for networks. *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 2016 :855–64

42. Mikolov T, Sutskever I, Chen K, Corrado GS, and Dean J. Distributed representations of words and phrases and their compositionality. Advances in neural information processing systems 2013; 26

43. Dong Y, Chawla NV, and Swami A. metapath2vec: Scalable representation learning for heterogeneous networks. *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*. 2017 :135–44

44. Wu Z, Pan S, Chen F, Long G, Zhang C, and Yu PS. A comprehensive survey on graph neural networks. IEEE transactions on neural networks and learning systems 2020; 32:4–24

45. Li Q, Han Z, and Wu XM. Deeper insights into graph convolutional networks for semi-supervised learning. *Proceedings of the AAAI conference on artificial intelligence*. Vol. 32. 1. 2018

46. Dwivedi VP, Joshi CK, Luu AT, Laurent T, Bengio Y, and Bresson X. Benchmarking graph neural networks. Journal of Machine Learning Research 2023; 24:1–48

47. Gasteiger J, Weißenberger S, and Günnemann S. Diffusion improves graph learning. Advances in neural information processing systems 2019; 32

48. Ma L, Rabbany R, and Romero-Soriano A. Graph attention networks with positional embeddings. *Pacific-Asia conference on knowledge discovery and data mining*. Springer. 2021 :514–27

49. Maas AL, Hannun AY, and Ng AY. Rectifier Nonlinearities Improve Neural Network Acoustic Models. *Proceedings of the 30th International Conference on Machine Learning (ICML)*. Vol. 30. 2013

50. Clevert DA, Unterthiner T, and Hochreiter S. Fast and accurate deep network learning by exponential linear units (elus). arXiv preprint arXiv:1511.07289 2015; 4:11

51. Kingma DP and Ba J. Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980 2014

52. Rousseeuw PJ. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. Journal of computational and applied mathematics 1987; 20:53–65

53. Olsson T, Ericsson M, and Wingkvist A. To automatically map source code entities to architectural modules with Naive Bayes. Journal of Systems and Software 2022; 183:111095

54. Laser MS, Medvidovic N, Le DM, and Garcia J. ARCADE: an extensible workbench for architecture recovery, change, and decay evaluation. *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE '20. ACM, 2020 :1546–50. DOI: 10.1145/3368089.3417941

55. Wen Z and Tzerpos V. An effectiveness measure for software clustering algorithms. *Proceedings. 12th IEEE International Workshop on Program Comprehension, 2004.* IEEE. 2004 :194–203

56. Lutellier T, Chollak D, Garcia J, Tan L, Rayside D, Medvidović N, and Kroeger R. Measuring the impact of code dependencies on software architecture recovery techniques. IEEE Transactions on Software Engineering 2017; 44:159–81

57. Le DM, Behnamghader P, Garcia J, Link D, Shahbazian A, and Medvidovic N. An empirical study of architectural change in open-source software systems. *2015 IEEE/ACM 12th working conference on mining software repositories*. IEEE. 2015 :235–45

58. Garcia J, Le D, Link D, Behnamghader AP, Ortiz EF, and Medvidovic N. An empirical study of architectural change and decay in open-source software systems. USC-CSSE, Tech. Rep 2014

59. Hubert L and Arabie P. Comparing partitions. Journal of classification 1985; 2:193–218

60. Mann HB and Whitney DR. On a test of whether one of two random variables is stochastically larger than the other. The annals of mathematical statistics 1947 :50–60

61. Cliff N. Dominance statistics: Ordinal analyses to answer ordinal questions. Psychological bulletin 1993; 114:494

62. Jabeen R, Ericsson M, Nordqvist J, and Wingkvist A. Reproduction package for GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery. Zenodo. Version 1.1.0. 2026. DOI: 10.5281/zenodo.19534384

63. RARE Technologies. Gensim: Topic Modelling for Humans. https://radimrehurek.com/gensim/. Python library. 2026